

CERTIFICATE OF EXCELLENCE

This is to certify that the research paper
*"Classifying Images/PDFs using Computer
Vision Techniques"*, submitted by Mr..

Ashish Gupta

This activity is awarded by:

has been awarded with the **"Extremely
Useful"** category.



Global IME Bank

ग्लोबल आइएमई बैंक लि.

सबैका लागि बैंक

Certificate ID: GIBL-1599456000

This certificate is digitally issued and can be verified at <https://hackathon.gibl.com.np/certificates>



DocLipi

An Intelligent Document Classifier and OCR Solution

Submitted To,

Global IME Bank

Kamaladi, Kathmandu, Nepal

Submitted By,

Ashish Gupta

Rubina Dangol Maharjan

Saniti Shakya

Submission Date,

2025-04-03

Abstract

The increasing need for automated document processing has led to the development of AI-driven solutions for classification and text extraction. DocLipi is a desktop-based system designed to classify official Nepali documents, such as Citizenship Cards, PAN Cards, Passports, and Bank Forms, using deep learning-based visual classification and optical character recognition (OCR). The project leverages ResNet-50 and DenseNet-121 architectures to classify documents based on visual features rather than textual content, ensuring accurate categorization even for handwritten or stamped documents.

For OCR, both Tesseract and EasyOCR are evaluated for their performance in recognizing Nepali and English text. The classification and OCR models are deployed via Sockets and REST API (Flask/FastAPI), while the desktop application is developed using JavaFX with MySQL as the backend database. Multi-page PDF documents are processed using Apache PDFBox, converting each page into an image before classification and OCR. Standard evaluation metrics such as accuracy, F1-score, confusion matrix, Character Error Rate (CER), and Word Error Rate (WER) are used to assess model performance.

By automating document classification and information extraction, DocLipi aims to enhance efficiency in banking and administrative sectors. The proposed solution demonstrates the viability of lightweight, localized AI solutions for document management in banking systems.



Contents

1. Introduction	3
2. Problem Description	3
3. Related Works.....	4
4. Proposed Methodology	
a. Document Classification	5
b. Character Recognition	9
c. Desktop Application	11
5. Implementation Mechanism	
a. Software Section	15
b. User Experience	16
6. Future Enhancements	18
7. Reference	19

Introduction

DocLipi is an automated document management system designed to streamline the classification and digitization of standardized Nepali documents, including Citizenship Cards, Passports, PAN Cards, and Account Opening Forms. Built for banking environments, the system combines computer vision and multilingual OCR to address the challenges of manually organizing and extracting critical text from unclassified PDFs and images. By leveraging pretrained convolutional neural networks (ResNet-50 and DenseNet-121) for visual classification and hybrid OCR engines (Tesseract and EasyOCR) for text extraction, DocLipi automates the end-to-end workflow—from document upload through secure MySQL storage. The JavaFX-based desktop application ensures user-friendly interaction, while REST APIs bridge Python-based AI models with the frontend, offering a scalable and privacy-focused solution tailored to Nepal's unique document standards.

Problem Description

In traditional banking systems, a significant challenge arises from the unstructured storage of customer documents. Banks receive numerous documents, including Citizenship Cards, PAN Cards, Passports, and Bank Registration Forms, which are often stored in an unclassified and disorganized manner. This lack of proper categorization creates difficulties in tracking customer records, retrieving necessary information for legal and administrative processes, and leveraging customer data for business insights and revenue generation. Additionally, manual data entry is a time-consuming and error-prone task, leading to inefficiencies in banking operations. To address these issues, an automated document classification and OCR-based system is required to digitize and systematically organize customer records, ensuring fast retrieval and streamlined processing.

Key Problems Faced by Banks

- **Unorganized Document Storage:** Customer documents are stored without proper categorization, making retrieval slow and inefficient.
- **Difficulty in Customer Tracking:** Banks struggle to track customer details efficiently, impacting customer management and business strategies.
- **Legal and Administrative Challenges:** In legal matters, finding relevant documents quickly is a challenge, delaying legal proceedings.
- **Manual Data Entry Burden:** Entering data manually is time-consuming, error-prone, and inefficient.

Related Works

Bakkali et al. (2022) proposed a cross-modal deep learning framework that fuses features from document images and their corresponding OCR-extracted text. Their architecture utilizes convolutional neural networks such as NasNetLarge and Inception-ResNet-v2 to capture visual structures, while textual content is processed using static (GloVe, FastText) and dynamic (BERT) word embeddings. The text is first extracted using Tesseract OCR, then passed through a GRU network or directly to BERT for semantic representation. They demonstrate that late fusion of visual and textual features significantly enhances classification performance. Their method outperforms previous state-of-the-art approaches on the RVL-CDIP dataset, achieving a classification accuracy of 97.05%, highlighting the effectiveness of joint visual-textual representation learning.

Afzal et al. (2017) conducted a comprehensive evaluation of very deep convolutional neural network (CNN) architectures for document image classification, aiming to significantly reduce classification errors. They examined state-of-the-art models including AlexNet, VGG-16, GoogLeNet, and ResNet-50, and explored different weight initialization strategies, namely training from scratch, transfer learning from ImageNet, and pretraining on a large document dataset (RVL-CDIP). Their experiments showed that pretraining on domain-specific data (i.e., document images) consistently outperformed other methods. On the RVL-CDIP dataset of 400,000 document images across 16 classes, VGG-16 achieved a top accuracy of 90.97%, surpassing previous benchmarks. Additionally, when evaluated on the smaller Tobacco-3482 dataset, their approach achieved over 91% accuracy, with a relative error reduction of more than 60% compared to earlier methods.

Zoph et al. introduced a novel approach to automate convolutional neural network (CNN) design using Neural Architecture Search (NAS) with reinforcement learning. Their framework learns a convolutional cell on the smaller CIFAR-10 dataset and then transfers it to larger-scale tasks like ImageNet classification. They designed a new search space called the NASNet search space, allowing scalable architecture transfer by stacking learned convolutional cells. The controller RNN samples child network architectures, evaluates their validation accuracy, and updates itself via policy gradients. The best-found architecture, NASNet-A, achieved 2.4% error on CIFAR-10 and 82.7% top-1 accuracy on ImageNet, outperforming state-of-the-art human-designed models while using significantly fewer FLOPs. Their approach demonstrates that learned architectures can generalize across datasets and computational budgets, even excelling in object detection when transferred to Faster-RCNN, achieving 43.1% mAP on COCO. The research highlights the potential of automated architecture design in producing competitive and efficient deep learning models across tasks and platforms.

Proposed Methodology

The proposed system leverages Artificial Intelligence (AI) and Optical Character Recognition (OCR) to efficiently classify and extract text from banking documents. The project is divided into three key components:

- **Document Classification:** Utilizes ResNet and DenseNet architectures with transfer learning to classify scanned documents.
- **Character Recognition:** Implements Tesseract and EasyOCR to extract textual information from classified documents.
- **Desktop Application:** Provides a user-friendly interface for document uploading, classification, and retrieval, integrating both CNN and OCR components.

1. Document Classification

Document Classification is the process of categorizing scanned documents into predefined classes using Artificial Intelligence (AI) and Deep Learning. Traditional methods rely on manual sorting, which is inefficient and prone to errors. By leveraging Convolutional Neural Networks (CNNs), specifically ResNet and DenseNet, our system automates document classification with high accuracy. The model is trained to recognize different banking documents, such as Citizenship Cards, PAN Cards, Passports, and Bank Registration Forms, reducing manual effort and improving data organization. Through transfer learning, feature extraction, and fine-tuning, the system ensures efficient classification, paving the way for seamless integration with Optical Character Recognition (OCR) for text extraction.

I. AI Concepts Used in Classification

- **Convolutional Neural Network (CNN):** A deep learning architecture designed to process grid-like data (e.g., images) using convolutional layers to detect spatial patterns.
- **Transfer Learning:** Reusing a pre-trained model (e.g., ResNet trained on ImageNet) and adapting it to a new task (document classification) by retraining only select layers.
- **Data Augmentation:** Artificially expanding the dataset by applying transformations (rotations, shifts) to improve model robustness.



- **Hyperparameters:** Configurable settings (e.g., learning rate, batch size) that control model training.

II. Model Development

Data Collection

- Dataset:
 - Classes: Citizenship, Passport, PAN Card, Account Opening Form.
 - Samples: 10 images per class, collected via web scraping and local volunteers.
 - Ethical Compliance: Data anonymized to remove sensitive personal information.
- Public Datasets: Supplementary images sourced from Kaggle's document datasets (cite source).
- Hackathon Dataset: We will utilize the dataset provided during the hackathon to train our AI Models.

Data Preprocessing

1. Resizing: All images standardized to 512×512 pixels to ensure uniform input dimensions.
2. Normalization: Pixel values scaled to $[0, 1]$ by dividing by 255.
3. Augmentation: Applied to mitigate overfitting:
4. Data Augmentation: Applying rotation, flipping, and brightness adjustments to enhance model robustness.
5. Label Encoding: Assigning unique labels to each document type for supervised learning.

Model Selection

The project utilizes two pre-trained models for classification:

ResNet (Residual Network):

ResNet, short for Residual Network, is a deep convolutional neural network architecture renowned for its innovative use of residual blocks to address the vanishing gradient problem in deep networks. Introduced by He et al. in 2015, ResNet-50 (a 50-layer variant) employs



skip connections that bypass one or more layers, allowing gradients to propagate directly through these shortcuts during training. This design enables the training of extremely deep networks (e.g., 50, 101 layers) without degradation in accuracy. In the context of document classification, ResNet's hierarchical feature extraction excels at recognizing visual patterns such as color schemes, logos, and structural layouts inherent to standardized Nepali documents. By freezing initial layers (pre-trained on ImageNet) and retraining later layers on document-specific data, ResNet efficiently adapts to the target task through transfer learning, ensuring robust performance even with limited labeled samples.

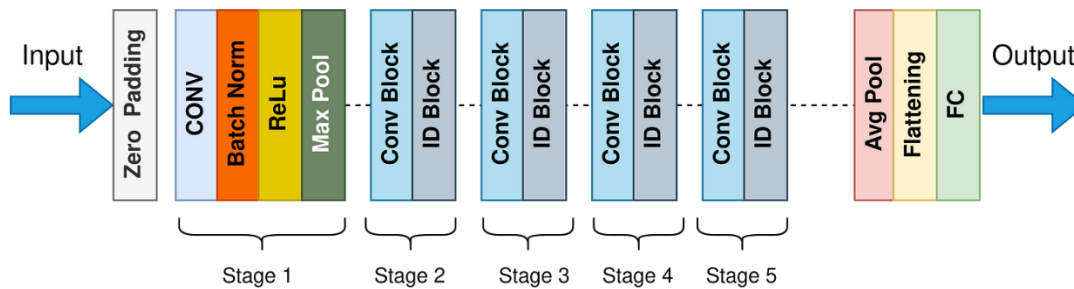


Fig: ResNet50 Architecture

- Uses residual connections to prevent vanishing gradients.
- Efficient in deep networks with minimal loss of information.
- Suitable for large-scale document classification.

DenseNet (Densely Connected Convolutional Network):

DenseNet, or Densely Connected Convolutional Network, distinguishes itself through dense blocks, where each layer receives feature maps from all preceding layers and passes its own outputs to all subsequent layers. This architecture, proposed by Huang et al. in 2016, promotes feature reuse, reduces redundancy, and enhances gradient flow across the network. For instance, DenseNet-121 (121 layers) concatenates feature maps from earlier layers, enabling the model to learn complex visual hierarchies with fewer parameters compared to ResNet. In document classification, this property helps capture subtle distinctions between Nepali document types (e.g., the blue hue of passports vs. the green citizenship cards) by aggregating multi-scale features. DenseNet's compact design and efficient parameter usage make it ideal for scenarios requiring high accuracy without excessive computational overhead, aligning well with the project's focus on lightweight, scalable solutions.

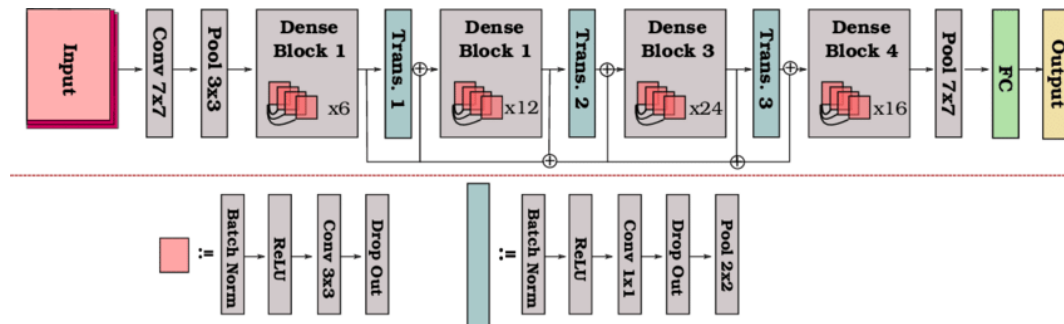


Fig: ResNet112 Architecture

- Uses dense connections to improve feature propagation.
- Requires fewer parameters compared to ResNet.
- Helps in better feature reuse for classification.

Model Training & Evaluation

- **Dataset Splitting:** 80% of data for training, 10% for validation, and 10% for testing.
- **Loss Function:** Cross-Entropy Loss to measure classification accuracy.
- **Optimizer:** Adam Optimizer for adaptive learning rate adjustments.
- **Evaluation Metrics:** Accuracy, Precision, Recall, and F1-score to assess model performance.

Hyperparameter Tuning

- **Batch Size Optimization:** Testing different batch sizes to improve convergence.
- **Learning Rate Adjustment:** Using learning rate schedulers to optimize training speed.
- **Epoch Tuning:** Selecting the optimal number of training epochs to prevent overfitting.
- **Dropout Regularization:** Preventing overfitting by randomly disabling neurons during training.



Model Deployment

- **Local Deployment:** The trained model will run locally on GPU-enabled devices using CUDA processing units for efficient inference.
- **Backend Framework:** The model will be hosted using Flask, exposing a REST API for classification queries.
- **Virtual Environment:** The deployment will use Python with necessary deep learning libraries installed in an isolated environment.
- **Java Integration:** The desktop application (Java-based) will send classification requests to the Flask server, which will process the image and return results in real time.

2. Character Recognition

Character Recognition (OCR) is the process of converting scanned document images into machine-readable text. In this project, OCR is applied to extract text from classified banking documents in both **English** and **Nepali** languages. The OCR Models proposed to use the software are:

- Tesseract OCR
- EasyOCR

Tesseract OCR

Tesseract is an open-source OCR engine widely used for text recognition in scanned documents. It supports multiple languages, including English and Nepali, making it a suitable choice for this project.

- Works using LSTM (Long Short-Term Memory) networks for improved accuracy.
- Performs well with clean and high-resolution text images.
- Can be fine-tuned for Nepali text recognition with custom training datasets

EasyOCR

EasyOCR is a deep-learning-based OCR library that supports over 80 languages, including Nepali. It leverages pre-trained deep learning models for text detection and recognition.

- Uses PyTorch-based CNN architectures for text extraction.
- Can handle noisy, distorted, and multi-font text better than traditional OCR methods.
- Faster and more efficient for real-time text extraction



I. Preprocessing Techniques

To optimize OCR accuracy, raw images undergo three preprocessing steps:

1. **PDF-to-Image Conversion:**

PDFs are split into individual pages using Apache PDFBox and rendered as 300 DPI images to preserve text clarity.

2. **Denoising:**

OpenCV's Gaussian Blur reduces scan artifacts (e.g., dust, folds), while median filtering removes salt-and-pepper noise.

3. **Binarization:**

Images are converted to black-and-white using Otsu's thresholding, enhancing text-background contrast for OCR engines.

II. Performance Metrics

OCR accuracy is quantified using:

- **Character Error Rate (CER):**

$$CER = \frac{\text{Number of Insertions} + \text{Deletion} + \text{Substitution}}{\text{Total Character in Ground Truth}}$$

- **Word Error Rate (WER):**

$$WER = \frac{\text{Number of Incorrect Words}}{\text{Total Word in Ground Truth}}$$

III. Integration Workflow

- OCR processing is handled on a Flask REST API.
- Extracted text is sent to the JavaFX desktop application using JSON payloads.
- OCR output includes recognized text and bounding boxes for structured data representation.

3. Desktop Application

The JavaFX-based **DocLipi** application serves as the user interface for document upload, classification, OCR processing, and database management. Built with modularity in mind, it integrates Python-based AI models via REST APIs while ensuring secure data handling. Below is the implementation breakdown:

- **Frontend (Client):** JavaFX GUI for document upload, results display, and user interactions.
- **Backend (Server):** Python Flask API hosting ResNet/DenseNet classifiers and OCR pipelines.
- **Database:** MySQL for storing extracted text, document metadata, and file paths.

I. Application Features

Multi-format Document Upload

- Supports PDF and image (JPG, PNG, TIFF) uploads.
- Uses Apache PDFBox to convert multi-page PDFs into individual images for processing.

AI-based Document Classification

- Utilizes ResNet/DenseNet to classify documents (Citizenship, PAN Card, Passport, Bank Form).
- Sends document images to a Flask/FastAPI server for classification via REST API.

OCR Processing in Nepali & English

- Uses Tesseract/EasyOCR to extract text from documents.
- Displays OCR-extracted text with confidence scores.
- Overlays detected text bounding boxes on the document preview.

Database Storage & Retrieval

- Extracted data is stored in a MySQL database.
- Enables searching and filtering of stored documents based on text metadata.

User Authentication & Role-based Access

- Secure login system for different user roles (e.g., Admin, Bank Staff).
- Ensures restricted access to classified data.



Chat-based Document Query (AI-powered)

- Users can query documents in natural language (e.g., “Show me Ram Bahadur’s PAN Card”).
- AI-driven search retrieves and displays relevant documents & metadata.

Interactive Document Viewer

- Displays original documents with overlaid OCR text regions.
- Provides options to correct OCR errors manually.

Export & Report Generation

- Allows exporting extracted text in structured formats like CSV, JSON, or PDF reports.
- Enables easy integration with other banking systems.

II. Application Architecture

1. **JavaFX GUI:**

- **SceneBuilder:** Drag-and-drop UI design for intuitive document management.
- **ControlsFX:** Enhanced UI components (e.g., file upload dialogs, progress bars).

2. **REST API Integration:**

- **Apache HttpClient:** Sends document images to the Flask server for classification/OCR.
- **GSON:** Parses JSON responses (e.g., extracted text, bounding boxes).

3. **PDF/Image Handling:**

- **Apache PDFBox:** Converts PDFs to images (300 DPI) and vice versa.
- **Java AWT ImageIO:** Processes JPEG/PNG uploads.

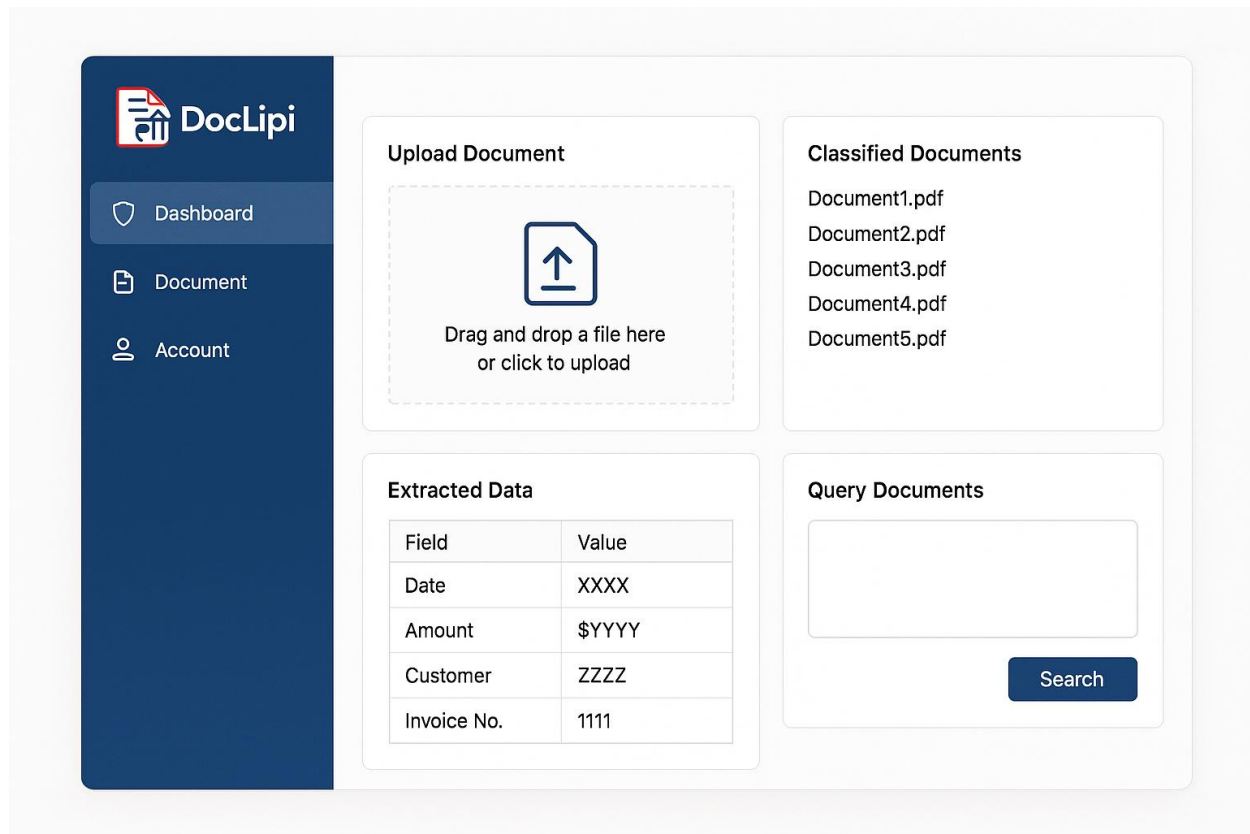


Fig: Concept Software Application UI

III. Workflow

1. Document Upload:

- Users upload PDFs/images via a JavaFX FileChooser.
- Multi-page PDFs are split into individual images using PDFBox.

2. Classification & OCR:

- Images are sent to the Flask API as base64 strings.
- The backend returns JSON with document type, extracted text, and confidence scores.

3. Results Display:

- Extracted text is shown in a scrollable TextArea.

- Bounding boxes overlaid on images (using Canvas) highlight OCR-detected regions.

4. **Database Storage:**

- Structured text and file paths are inserted into MySQL via JDBC.

IV. Database

The MySQL database serves as the central repository for storing user information submitted through the bank account opening form. Java Database Connectivity (JDBC) facilitates seamless communication between the Java-based desktop application and the database, ensuring efficient data storage and retrieval. The database securely stores essential user details, including the account holder's full name (first, middle, surname), guardian's name and relationship (if applicable), date of birth (both in B.S. and A.D.), nationality, gender, educational qualification, marital status, and comprehensive contact details such as residential phone number, mobile number, and email. Additionally, both permanent and current addresses are stored, including district, municipality, ward number, street, house number, province, and country. Alongside this structured data, the database maintains secure file paths to the documents submitted with the form, ensuring all supporting materials—such as Citizenship, PAN Card, Passport, or any other required document—are stored in a PDF format as a single file per user. The system does not store any additional information beyond what is explicitly provided in the account opening form, ensuring streamlined and secure document management within the banking system.

ER Diagram

The provided Entity-Relationship (ER) diagram represents the database schema for a bank document processing system. It consists of multiple entities, including Bank User, Document, OCR Text, Employee, and Global IME (Branch), each with unique attributes and relationships. The Bank User entity stores user details and is linked to the Document entity, which contains scanned documents. These documents undergo OCR processing, generating extracted text with a confidence score. The Employee entity represents bank employees responsible for processing documents, and they belong to specific branches under the Global IME entity. The relationships between these entities ensure secure document handling, user-account linkage, and structured OCR text extraction, making the system efficient for document classification and information retrieval.

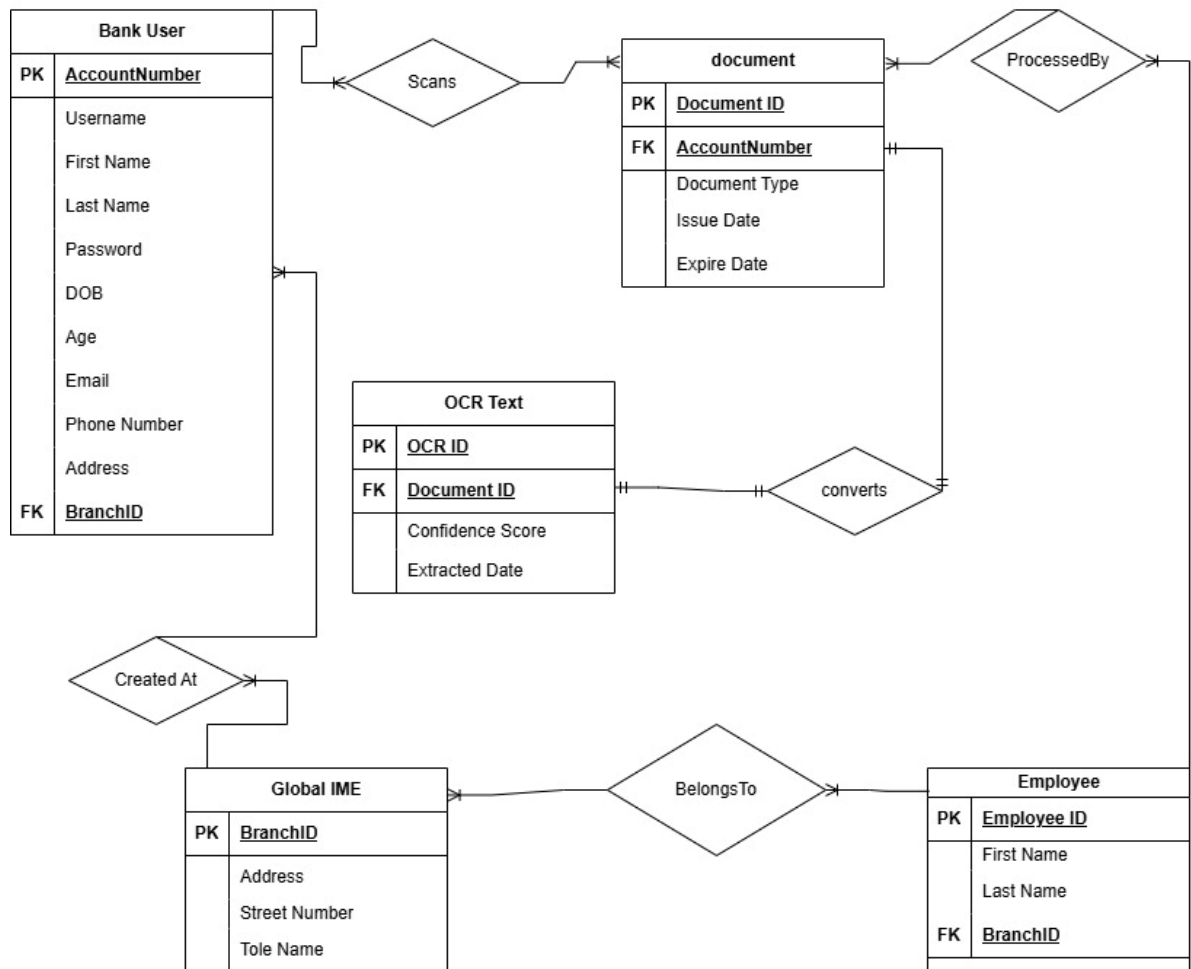


Fig: ER Diagram of Database

Implementation Mechanism

The DocLipi application serves as the central hub for document classification, OCR processing, and database management. Designed primarily as an Android application, it enables bank employees to streamline the document handling process efficiently. The application integrates AI models for document classification and OCR through a backend server and ensures secure data storage in a structured MySQL database. With an intuitive user interface, it simplifies the process of extracting, organizing, and retrieving customer data while maintaining accuracy and security.

Software Sections Overview

Our desktop application consists of several key sections designed to streamline the process of document uploading, data extraction, and database management. Below is an outline of the core features:

1. **Document Upload Section:** The document upload section allows users to upload both PDF and image files (JPEG, PNG, TIFF) into the system. The user can easily drag and drop the files or use a file browser to select the documents. Once uploaded, the system automatically converts multi-page PDFs into individual images using Apache PDFBox and sends the documents for classification and OCR processing.
2. **AI Model Classification Result Section:** After the document upload, the AI-based classification model (ResNet/DenseNet) processes the documents, classifying them into categories such as Citizenship Card, PAN Card, Passport, or Bank Form. In this section, users can view the classified documents along with their respective types clearly displayed, ensuring that the classification is accurate. The system visually distinguishes the documents with clear labels, allowing the bank staff or user to verify the classification before proceeding with OCR processing.
3. **Data Extraction & Modification Section:** After the classification step, the OCR engine (Tesseract/EasyOCR) extracts relevant data from the documents, including text fields like the name, date of birth, nationality, gender, and other important information. This data is displayed in an editable form, allowing users to review and modify the extracted information as necessary. Once satisfied with the data, users can apply the changes and store them in the MySQL database, ensuring that the most accurate and up-to-date details are retained.



4. **Database Search Section:** This section provides a powerful search tool, enabling users to quickly locate records in the database. The search functionality is designed to be interactive and efficient, allowing users to search by various unique fields such as name, date of birth, account number, and other relevant criteria. Users can filter and narrow down their search results based on multiple parameters to find the exact record they need.
5. **Chat-Based Document Query Section:** For ease of use, this section enables users to interact with the application using natural language queries. Users can ask questions such as "Show me Ram Bahadur's PAN Card," and the AI-driven chat system will process the request and retrieve the relevant document or data. This intuitive feature enhances the user experience, allowing users to interact with the system in a more conversational way.
6. **Main Dashboard Section:** The main dashboard provides an overview of the system's status and performance. It displays critical information such as the total number of documents and records stored in the database, the number of active users, and recently added data. The dashboard also provides insights into user activity and database health, offering a snapshot of the system's overall status and performance. This section is designed to keep users informed and ensure efficient management of the bank's data.

User Experience & Workflow

The implementation of DocLipi follows two distinct approaches, ensuring adaptability to different banking scenarios. These approaches define how documents are processed and stored in the system.

Approach 1: New Customer Document Processing

1. **Customer Submission:** A customer applying for a bank account fills out an account opening form and submits the necessary documents, including citizenship card, PAN card, passport, and other identity documents.
2. **Document Upload & Classification:** The bank employee scans and uploads all documents in PDF format into the application.
3. **Automated Document Classification:** The application processes the uploaded PDF and classifies individual pages using ResNet/DenseNet models into predefined categories (e.g., account form, citizenship card, PAN card, etc.).



4. **OCR Processing:**

- The main OCR operation is performed on the account opening form, extracting key details such as name, date of birth, contact information, and address.
- Supporting documents (citizenship card, PAN card, etc.) are stored and linked to the extracted data.

5. **Data Storage:** The extracted text and document metadata are stored securely in a MySQL database, allowing seamless access and retrieval.

6. **User Access & Querying:** The bank staff can later search and retrieve any customer's information through structured database queries or an AI-powered chat-based document search.

Approach 2: Processing Existing Bank Records

1. **Existing Document Upload:** Banks with a pre-existing database of customer records can upload previously scanned documents in bulk.

2. **OCR & Data Extraction:**

- The software performs OCR on the account opening forms to extract customer details.
- Extracted information includes names, unique identification numbers, and other relevant details.

3. **Intelligent Document Mapping:**

- Using extracted details from the form, the system intelligently matches supporting documents such as citizenship cards, PAN cards, and passports to their respective forms.
- This automated mapping process eliminates manual effort and organizes documents efficiently.

4. **Database Organization & Searchability:** Once mapped, all records are stored in a structured manner within the MySQL database, making it easy to retrieve any customer's full set of documents through query-based or AI-powered search.

Future Enhancements

DocLipi serves as a foundational step toward intelligent document processing and has the potential to be expanded into a fully automated, scalable, and secure document digitization system. By incorporating future enhancements, this project can further revolutionize digital document management in financial institutions, government offices, and corporate environments. DocLipi is a robust application with all the features necessary for the functionality of a Document Manager, still some of the advancements that can be made in the system are given below:

1. Hybrid Document Understanding Model

Develop a unified AI architecture combining vision transformers (ViT) and layout-aware OCR to jointly classify documents, extract text, and understand spatial relationships (e.g., mapping "Name" labels to adjacent text boxes). This would replace the current sequential pipeline, improving efficiency and accuracy for complex layouts.

2. Mobile Application for On-the-Go Scanning and Classification

Develop a mobile-friendly version of the system, allowing users to scan and classify documents directly from their smartphones. This would make the system more accessible and allow users to upload and retrieve documents anytime, anywhere.

3. Expanded Document Class Coverage

Currently, our system classifies four types of documents. A key future improvement is expanding the classification model to cover a broader range of legal and financial documents commonly required in banking.

References

- Bakkali, S., Ming, Z., Coustaty, M., & Rusiñol, M. (2020). Visual and textual deep feature fusion for document image classification. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2394–2403.
- Afzal, M. Z., Kölsch, A., Ahmed, S., & Liwicki, M. (2017, November). Cutting the error by half: Investigation of very deep cnn and advanced training strategies for document image classification. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR) (Vol. 1, pp. 883–888). IEEE.
- Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 8697–8710)
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4700–4708).
- EasyOCR. (n.d.). JaidedAI/EasyOCR: Ready-to-use OCR with 80+ languages supported. GitHub. Retrieved April 3, 2025, from <https://github.com/JaidedAI/EasyOCR>
- Tesseract OCR. (n.d.). Tesseract-ocr/tesseract: Tesseract Open Source OCR Engine. GitHub. Retrieved April 3, 2025, from <https://github.com/tesseract-ocr/tesseract>
- Apache PDFBox. (n.d.). A Java PDF library. The Apache Software Foundation. Retrieved April 3, 2025, from <https://pdfbox.apache.org/>
- OpenJFX. (n.d.). JavaFX: Cross-platform GUI framework for Java. Retrieved April 3, 2025, from <https://openjfx.io/>
- Flask. (n.d.). Flask web framework documentation. Pallets Projects. Retrieved April 3, 2025, from <https://flask.palletsprojects.com/en/stable/>
- MySQL Documentation. (n.d.). MySQL reference manual. Oracle. Retrieved April 3, 2025, from <https://dev.mysql.com/doc/>